Technische Universität Braunschweig

Institute of Operating Systems and Computer Networks

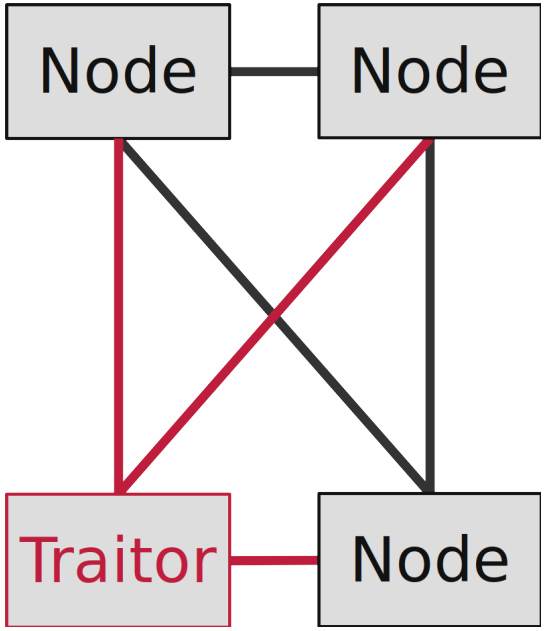
# Resilient Byzantine Fault-Tolerance

## Using Multiple Trusted Execution Environments

Markus Becker, July 15, 2021

# Byzantine Agreement

- Goal: Reach consensus across multiple machines
- Application: State-Machine Replication (SMR)
  - Agree & Order requests
  - Execute deterministic operation




**Byzantine Failures**

- Malicious party exhibits arbitrary behaviour

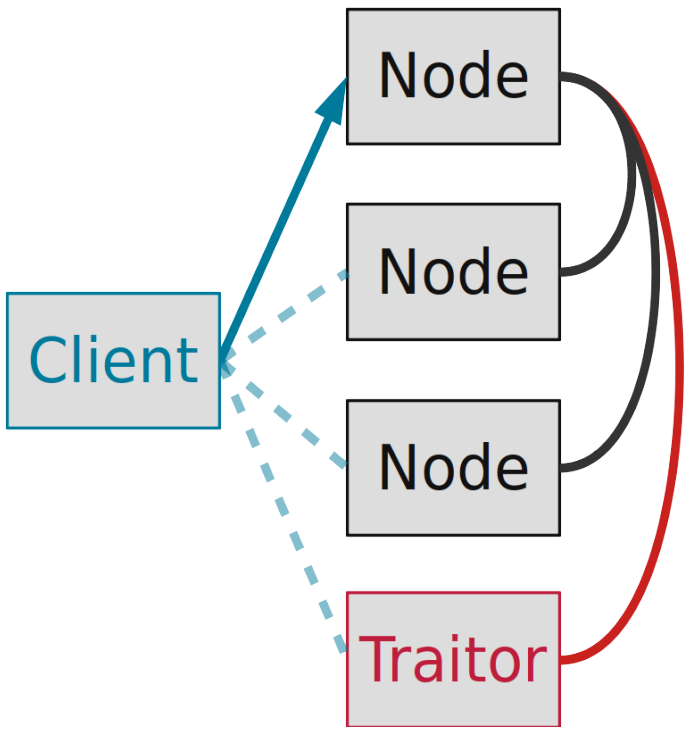
**Fault Tolerant Systems**

- Resilient against Byzantine Failures
- Operates correctly despite faults

# Byzantine Fault Tolerant (BFT) Protocols

Example protocol: PBFT M. Castro and B. Liskov '99

1. Replicated nodes act as servers
2. Client sends request
3. Communication rounds between replicas
   - Ordering and agreement
   - Requires $n \geq 3f + 1$ for $f$ faults
4. Client receives responses
5. Client performs voting on results



# BFT Protocol Deployment

Usage in permissioned Blockchains

- BFT for ordering and agreement
- BaaS: Nodes & Infrastructure by cloud provider
  - Amazon Managed Blockchain
  - Azure Blockchain Service
- Agreement using specific protocol

**But we do not want to have to trust the cloud provider!**

# Trusted Execution

Features of most TEEs:

- Execution of signed code on third parties machine
  - Local and remote attestation
- Confidentiality over host
  - Fully hardware-encrypted memory
- Reduced the chance of bugs
  - Small TCB

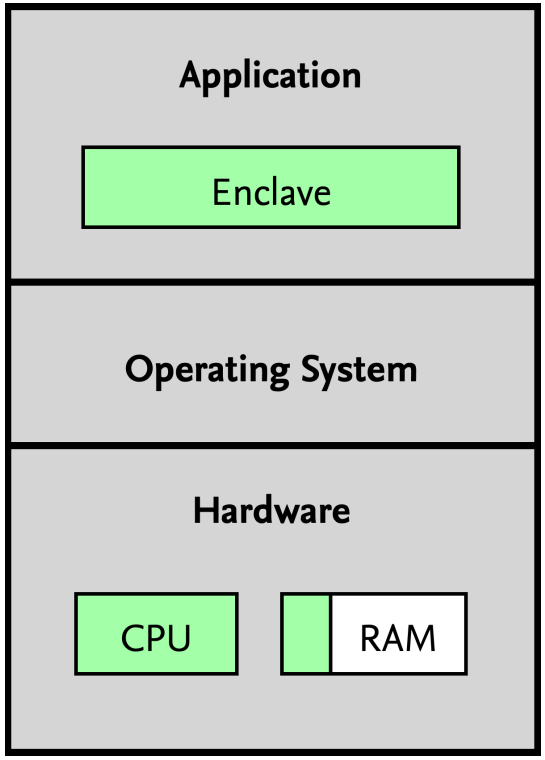
**But we still have to assume Byzantine failures!**

# Intel Secure-Guard-Extensions (SGX)

- Trusted Execution Environment
- Extension of x86
- Exclusively on Intel CPUs
- Transparently encrypted memory
- Ring-3 only execution
- (Remote) Attestation

---

**But enclaves can still contain bugs (or have other weaknesses)**



# Robustness

Reduce chance that practical BFT system enters irreparable state.

**Ordinary**

- Increase number of replicas
- Use safe programming strategies / frameworks
- Diversification (Lazarus'19)
  - Code & OS
  - Hardware
  - Passwords
  - Admins
- Rejuvenation

**Invasive**

- Trusted Execution
  - No golden bullet
  - Small *validated* TCB
  - Interface with OS needed
- Separation by functionality
  - Logically
  - Physically

# BFT Partitioning

**Naïve Solution:**

1. Put entire prototol in TEE
2. Use crypto to stop attacks on safety

**Problem:**

↯ Attacker in enclave ⇒ replica compromised: safety & liveness

**Solution:**

- Separate agreement protocol into partitions (compartments)
  - Increase robustness of individual enclave

↯ Attacker in enclave ⇒ liveness of replica

↯ Attacker in $f$ enclaves of same type ⇒ safety & liveness

# Thesis Goals

## Contribution: Combination of robustness features

- Trusted Execution (SGX)
- Separating compartments (SplitBFT)

## Goals

- Improve safety & resilience using TEEs
  - Tolerate up to $f$ faults per compartment type
- Keep confidentiality as long as possible
  - All execution compartment uncompromised

# Design

- Split PBFT into small protocol units for compartments
  - Efficient memory management (avoid copies)
  - Avoiding SGX overhead (minimize Ecalls/Ocalls)
  - Translating between untrusted Themis environment and minimal TCB
- Enclaves should be independent and not rely on each other
- Security-sensitive functions especially isolated
- Eliminating common shared local state

# Compartmentalization

## Splitting State-Machine-Replication

- SMR is often physically split into Clients and Replicas
- Replicas are logically split into Agreement and Execution (SOSP'03)

We recognise further opportunity to split:

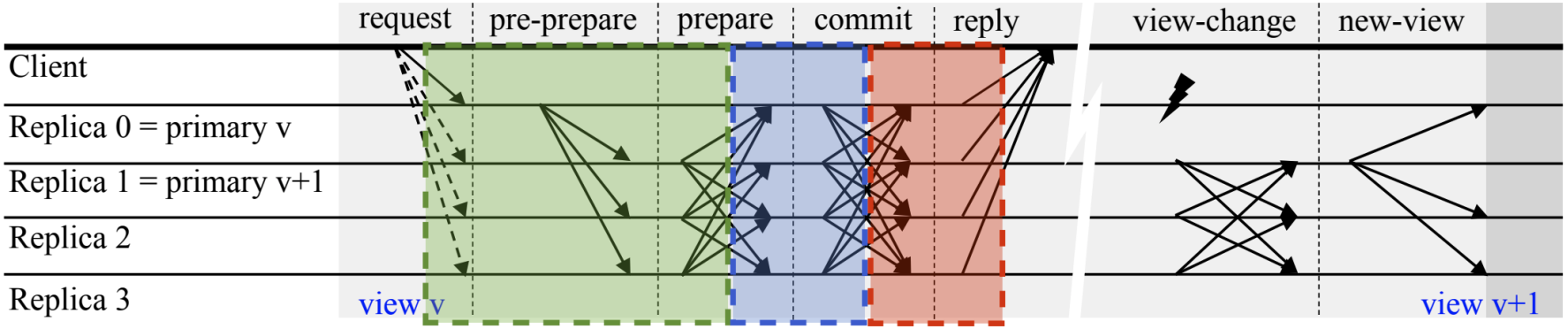
$$\text{BFT} \longrightarrow \text{Clients} + \text{Replicas}$$

$$\text{Replica} \longrightarrow \text{Agreement} + \textbf{Execution}$$

$$\text{Agreement} \longrightarrow \textbf{Preparation} + \textbf{Confirmation}$$

# SplitBFT: Normal Operation

Preparation, Confirmation, Execution




1. Collect messages in untrusted memory, liveness decisions outside
2. Verify signature (logically more participants than PBFT, origin-enclave-type matters!)
3. Follow PBFT-like operation

# Compartment's responsibilities

1. Preparation:
   - $\Leftarrow$ Receive requests, pre-prepares
   - $\square$ Order requests
   - $\Rightarrow$ Send pre-prepares, prepares
2. Confirmation:
   - $\Leftarrow$ Receive $\geq 2f$ prepares
   - $\Rightarrow$ Send commits
3. Execution:
   - $\Leftarrow$ Receive $\geq 2f + 1$ commits
   - $\square$ Execute requests
   - $\Rightarrow$ Send replies

## Asymmetric Cryptography

Messages are signed **by the enclave**, not the replica. This increases number of keys, but not number of signature-checks.

## Symmetric Cryptography

(Single-key) Symmetric encryption for requests and responses between client and execution enclaves.

# Checkpointing and View-Changes

**Design consideration:**

- Independence: no trust, no shared code
- Enclaves can not trust each other! (no *state* enclave)

**Garbage-collection & liveness:**

- Checkpoints
  - Application state only in Execution compartments
  - Allows removing old messages
- View-Changes
  - View *required* in all compartments
  - Cannot trust a global variable or even local "View-Compartment"

  $\Rightarrow$ New-View messages are broadcast to all compartments

# Themis

- IBR's own BFT Framework!
- Written in Rust: memory-safe, systems language
- Implemented protocols: PBFT, Railchain
- Implemented applications: Benchmark-Counter, *YCSB-KVS*[WIP]

# Implementation

1. Setup for Teaclave SGX SDK
2. Integration into Themis
   - Dependencies, dependencies, dependencies
3. Structured communication between TCB and Themis

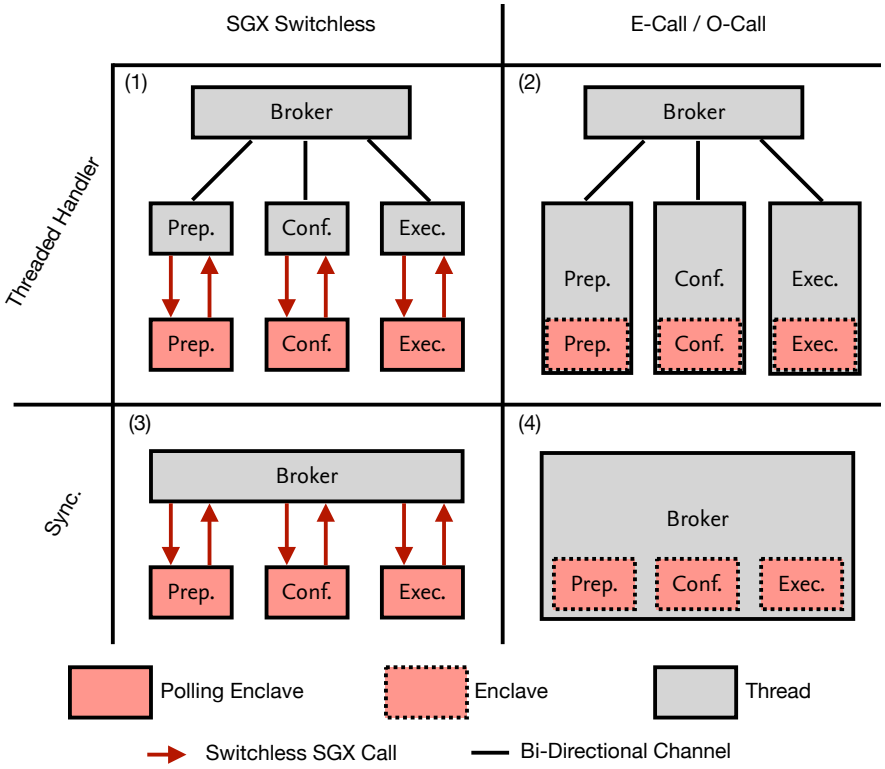
# Setup

- Rust `nightly-2021-02-17`
- Themis `@master`
- Teaclave SGX SDK >`1.1.3`
  - `1.1.3` incompatible with Themis
  - `master` incompatible with `SGX World`

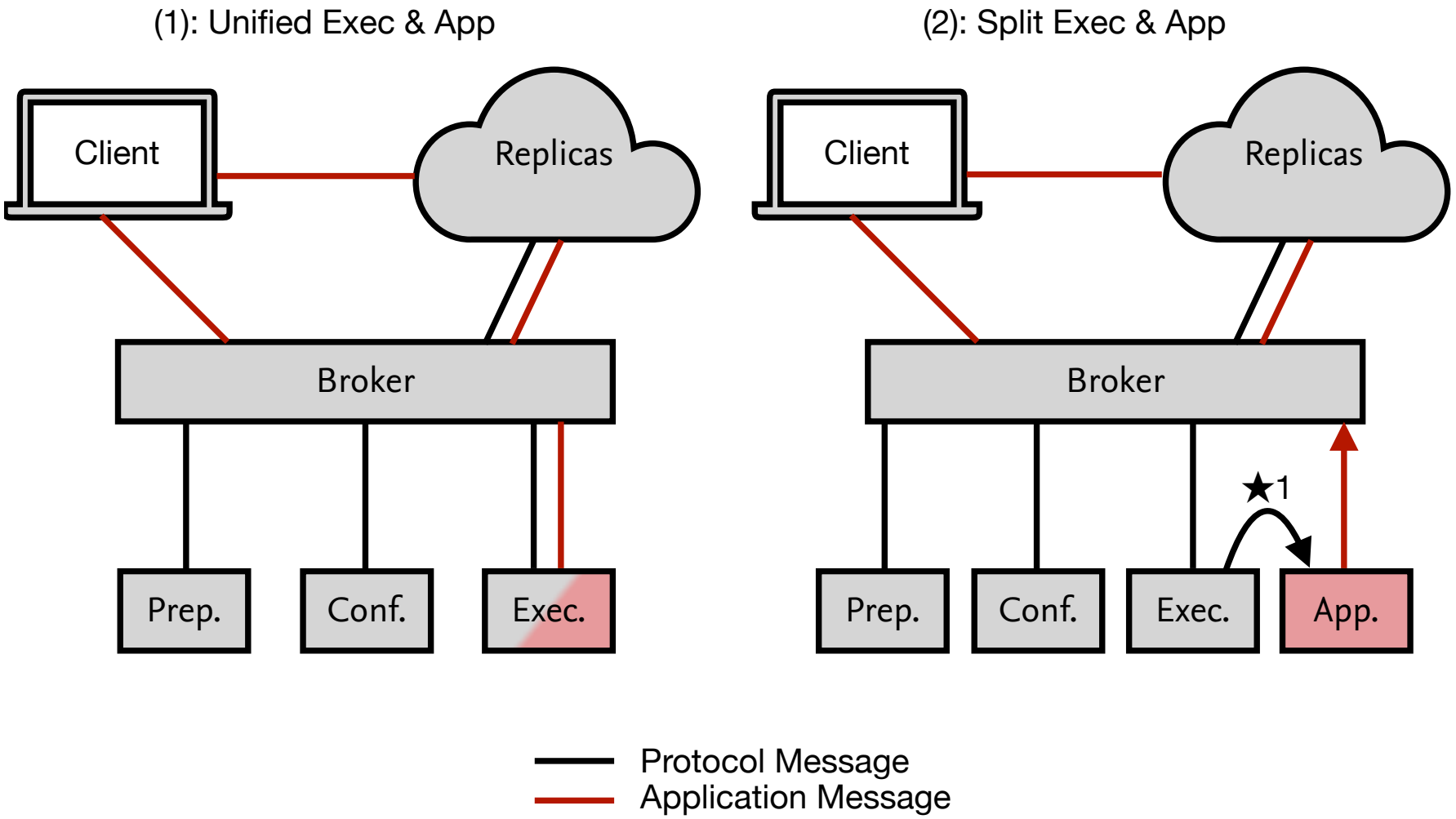
⇒ Own `SGX World` forks & docs + wiki
  - `bytes-sgx`, `ring-sgx`, `serde-sgx`, `msgpack-rust-sgx`

# SGX Broker Layer

- Compatible management structure for enclaves
- Efficient memory management
- Minimize SGX overhead
- Translation layer between Themis and TCB
- Deciding asynchronous/synchronous operation

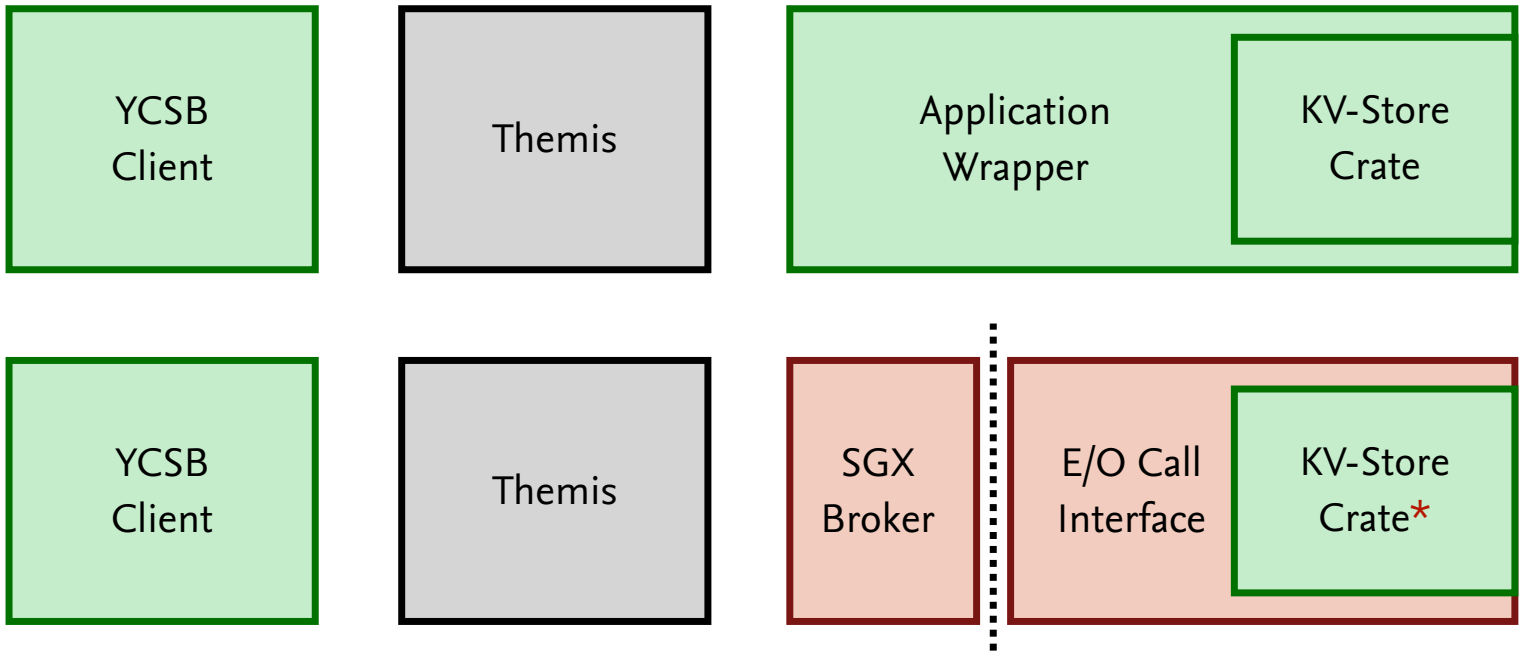


# Option: Execution & Application split



# Adaptation of KV-Store implementation

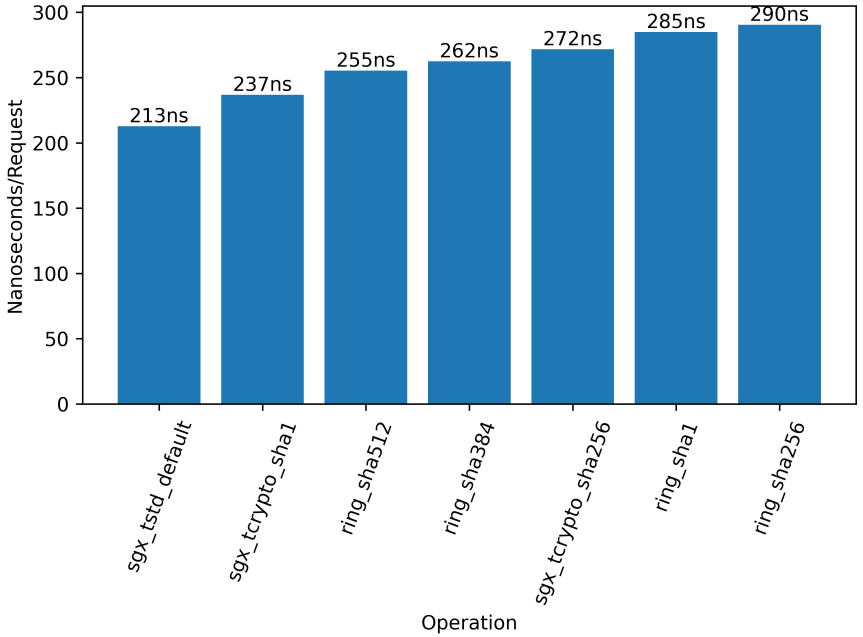
Writing TCB to allow swapping of non-SGX and SGX SMR without changing the client:




Wrapping for transparent interation with Themis is non-trivial

# Evaluation

- Benchmarking `#[no_std]` crates in SGX
- Deploying and measuring integration tests
- Measuring memory usage within enclaves during operation
- Measuring ECall/OCall abstraction overheads



# Conclusion

## Current Work

- Implement and contribute to SplitBFT protocol
- Show practical application of SGX and separation for BFT
- Extend and review Themis
- Measure and optimize overhead of added robustness